COMP1610 Coursework Report

Programming Enterprise Components

Bank App Project

Jack Jibb
001408490
Other Group Members:
Syed Ali Irtaza – 001414068
Anastasiia Pyslar – 001412071
Devanshi Pandya – 001426270
Poorva Motaval – 001449334

Submission Date: 24 March 2025

Contents

1 Design Documentation (Group Work) 3
1.1 Entity-Relationship Diagram . . . 3
1.2 Architecture Diagram . . . 3
1.2.1 Authentication & Session Management . . . 4
1.2.2 Student Management (CRUD) . . . 4
1.2.3 Account Management (CRUD) . . . 4
1.2.4 Transactions (Deposit, Withdraw, Transfer) . . . 4
1.2.5 Front-End Features . . . 5
1.2.6 RESTful API (JSON*) . . . 5
1.3 REST API Endpoints . . . 5

2 Command Line Application 6
2.1 Overview . . . 6
2.2 Project Structure . . . 6
2.3 How To Run . . . 6
2.4 Features and Command Menu . . . 6

3 Evaluation 9

4 Research (Individual) 9

5 Individual Reflection (Individual) 9

6 Appendix 10
6.1 Application Screenshots . . . 10
6.1.1 Web App Screenshots . . . 10
6.1.2 CLI Application Screenshots . . . 16
6.2 WildFly 33 Server Configuration . . . 17
6.2.1 Subsystems and Extensions . . . 17
6.2.2 Management and Access Control . . . 18
6.2.3 Server Interfaces and Socket Bindings . . . 18
6.2.4 Deployment and Monitoring . . . 18
6.2.5 Thread and Resource Management . . . 18
6.3 Source Code Structure . . . 18
6.4 References . . . 19

1 Design Documentation (Group Work)

1.1 Entity-Relationship Diagram

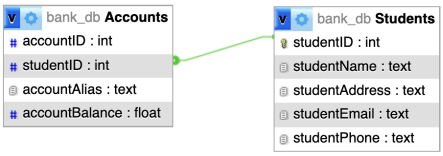

Figure 1: Entity Relationship Diagram for the Database

1.2 Architecture Diagram

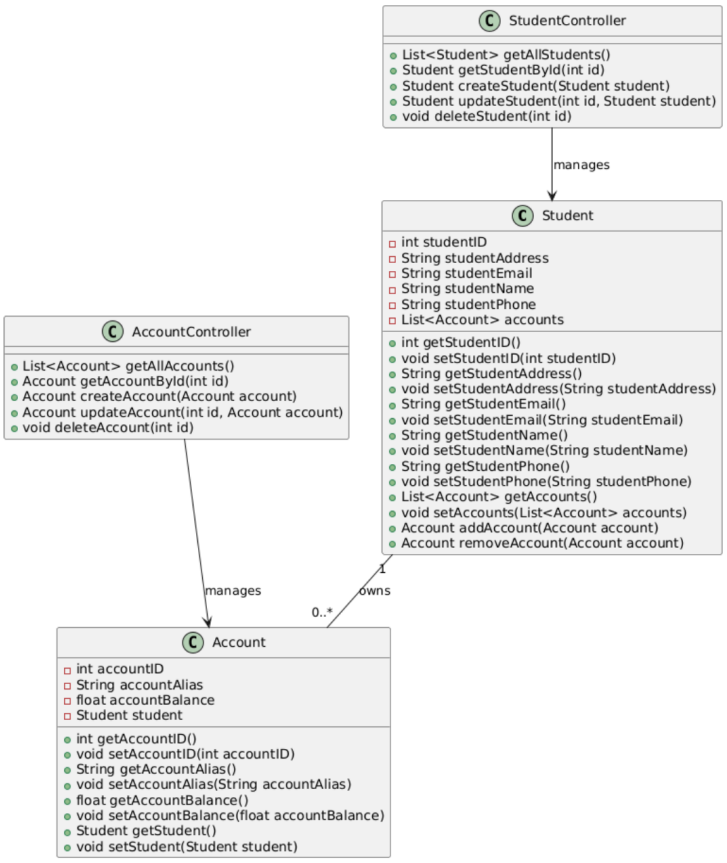

Figure 2: Overall Application Architecture

Feature List:

This section outlines the implemented features of the Banking Web Application, categorized by functionality. Each category includes a description and a self-assessment box (1 = Poor, 5 = Excellent) to reflect how well the features were implemented. See figures in appendix that items reference.

1.2.1 Authentication & Session Management

- **Student Login:** Dropdown menu for selecting and logging in a student Figure 4.
- **Session Cookies:** Username and student ID stored in cookies Figure 6.
- **Logout:** Clears cookies and session state Figure 7.

Implementation Rating (1–5): 4

1.2.2 Student Management (CRUD)

- Create student with form (name, email, phone, address) Figure 10.
- List all students (admin view) Figure 8.
- RESTful API: Create, Read, Update, Delete student Figure 11, Figure 9, Figure 10.
- Cascade delete verified upon student deletion.

Implementation Rating (1–5): 4

1.2.3 Account Management (CRUD)

- Create account (initial balance £0, linked via student ID from cookie) Figure 13.
- Conditional listing (all accounts or student-specific) Figure 12, Figure 15.
- Detailed view per account with deletion Figure 14.
- RESTful API: Create, Read, Update, Delete account Figure 21.

Implementation Rating (1–5): 5

1.2.4 Transactions (Deposit, Withdraw, Transfer)

- Deposit and withdraw via dynamic form Figure 18.
- Overdraft protection prevents excessive withdrawal.
- Transfer funds between accounts with Student validation Figure 16.
- TransactionResult page displays updated balances and outcome messages Figure 17.

Implementation Rating (1–5): 4

1.2.5 Front-End Features

- Dynamic page loading using JavaScript and AJAX.
- Button and form action binding via `main.js`.
- Confirmation prompts for sensitive actions (e.g., delete).

Implementation Rating (1–5): 5

1.2.6 RESTful API (JSON*)

- JSON-based API for students and accounts.
- Endpoints for CRUD operations.
- Special endpoints for deposit, withdraw, and transfer.

Implementation Rating (1–5): 3

**Note:* All JSON endpoints consume and produce data via `MediaType.APPLICATION_JSON`.

1.3 REST API Endpoints

List of all endpoints and the function they serve:

Student API

- **GET /api/students**:*getAllStudents()*
- **GET /api/students/student_id**:`getStudentByID(student_id)`
- **POST /api/students**:`createStudent(Student Entity)`
- **PUT /api/students**:`updateStudent(Student entity)`
- **DELETE /api/students/student_id**:`deleteStudent(student_id)`

Account API

- **GET /api/accounts**: *getAllAccounts()*
- **GET /api/accounts/account_id**: *getAccount(account_id)*
- **GET /api/accounts/studentID/student_id**: *getAccountsByStudentID(student_id)*
- **POST /api/accounts**: *createAccount(Account entity)*
- **PUT /api/accounts**: *updateAccount(Account entity)*
- **DELETE /api/accounts/account_id**: *deleteAccount(account_id)*
- **PUT /api/accounts/accounts_id**: *modifyAccountBalance(account_id)*
- **PUT /api/accounts/transfer**: *transferFunds(CustomJSON payload)*

2 Command Line Application

2.1 Overview

This is a Command-Line Interface (CLI) application for managing and their associated bank accounts. User can:

- Create/read/update/delete (CRUD) student and account data
- Make deposits, withdrawals, and transfers between accounts (legacy or new featured way)
- View all students and accounts in the system

2.2 Project Structure

The application uses a modular structure and communicates with external packages that handle requests to create, update, fetch, or delete records via exposed REST API.

```
├── Entities/
│   └── Contains data structures like Student, Account, Transaction
├── Requests/
│   ├── AccountBased/
│   │   └── Business logic, transactions, transfers
│   ├── Delete/
│   │   └── Delete operations
│   ├── Get/
│   │   └── Fetch data
│   ├── Post/
│   │   └── Create data
│   └── Update/
│       └── Update data
└── main.go
    └── CLI entry point
```

2.3 How To Run

In CLI directory, run `go run main.go`

2.4 Features and Command Menu

0 Exit

Exits the CLI application

1. Create Student

Prompts the user for student name, address, email, and phone, creates a new student.

Route: POST api/students

2. Create an Account for a Student

   Prompts for student ID, alias, and balance, create a new account for the mentioned student.

   Route: POST api/accounts/studentID/student_ID

3. Get a Student by ID

   Prompts for student ID, return the student who has specified ID

   Route: GET api/students/student_ID

4. Get an Account by ID

   Prompts for account ID, return the account that has specified ID

   Route: GET api/accounts/account_ID

5. Get Accounts by Student ID

   Retrieves all accounts linked to a given student ID

   Route: GET api/accounts/studentID/student_ID

6. Update a Student

   Allows updating name, address, email, and phone of a student

   Route: PUT api/students

7. Update an Account

   Updates account alias, balance and associated student ID

   Route: PUT api/accounts

8. Delete Student

   Removes Student by ID

   Route: DELETE api/students/student_ID

9. Delete an Account

   Removes Account by ID

   Route: DELETE api/accounts/account_ID

10. Process a Transaction (Deposit/Withdrawal)

    Prompts for account ID, operation type (deposit or withdraw), and amount. Offers two processing methods:

    Process Transaction (basic/legacy)

    Process Transaction Feature (During a withdrawal, the system checks if the selected account has sufficient funds. If not, it withdraws the available balance and continues with the student's other accounts to cover the remaining amount)

    Route: PUT api/accounts/account_ID

11. Process a Transfer between Two Accounts

    Transfers funds from one account to another. Requires both account IDs and student IDs. Offerstwo processing methods:

    ProcessTransfer (basic/legacy)

    ProcessTransferFeature (During a transfer, the system checks if the selected account has sufficient funds. If not, it withdraws the available balance and continues with the student's other accounts to cover the remaining amount)

    Route: PUT api/accounts/transfer

12. Show all Students

    Displays all student records in JSON format

    Route: GET api/students

13. Show All Accounts

    Displays all account records in JSON format

    Route: GET api/accounts

3 Evaluation

Everyone participated well in the development of the project, with a good distribution of workload. We had some issues getting off the ground due to configuration difficulties with the server files. This was remedied by working together to comb through each issue until we had synchronized environments. Everything started slowly since we were all busy with other coursework, but we managed to get together in time to finish the project successfully. We helped each other to uncover our weaknesses and improve them, for example, Anastasiia helped Jack with API functions and testing the functionality of the REST functions. We also had an issue with Wildfly server, where the debug mode would cause the server to hang on Persistence Manager loading indefinitely.

4 Research (Individual)

5 Individual Reflection (Individual)

6 Appendix

6.1 Application Screenshots

6.1.1 Web App Screenshots

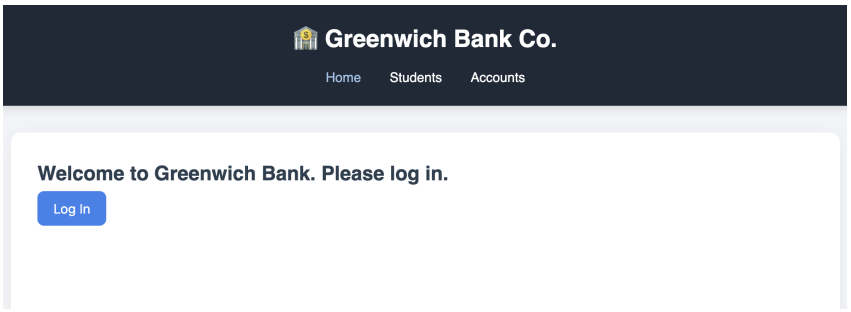

Figure 3: Initial Login Screen

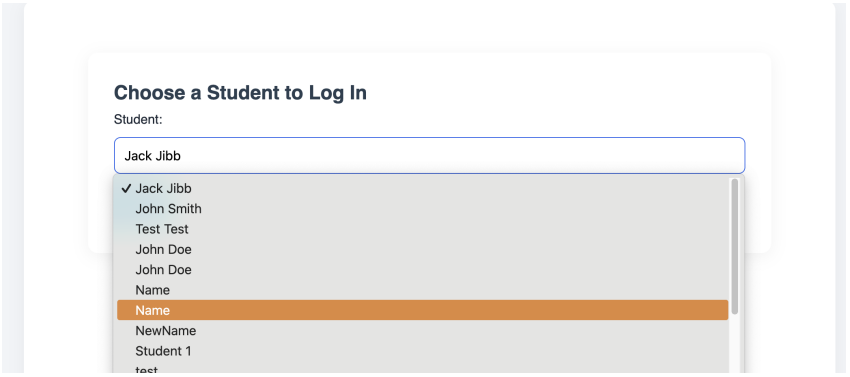

Figure 4: Ability to choose which Student to log in as

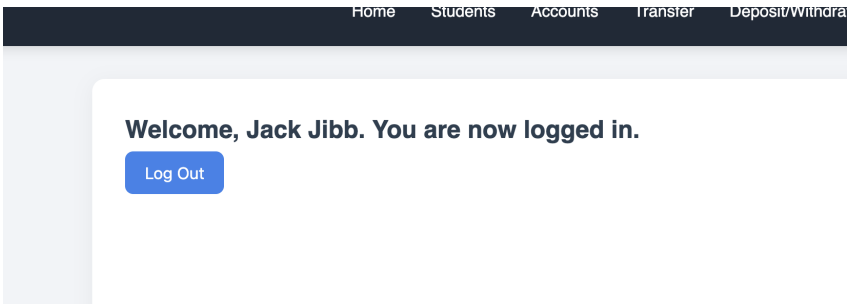

Figure 5: When logged in, the cookie allows the web page to display their name

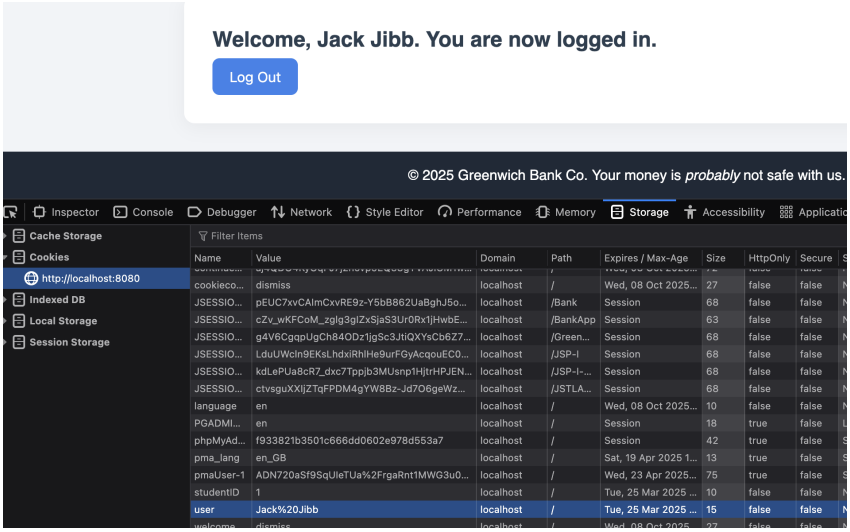

Figure 6: Cookie present when logged in

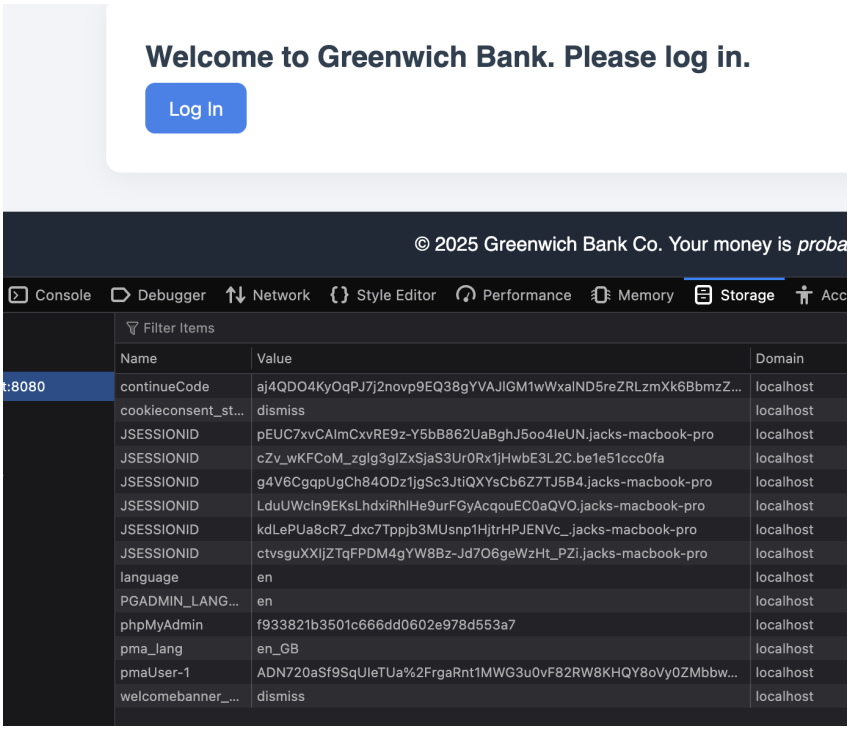

Figure 7: Cookie not present when not logged in

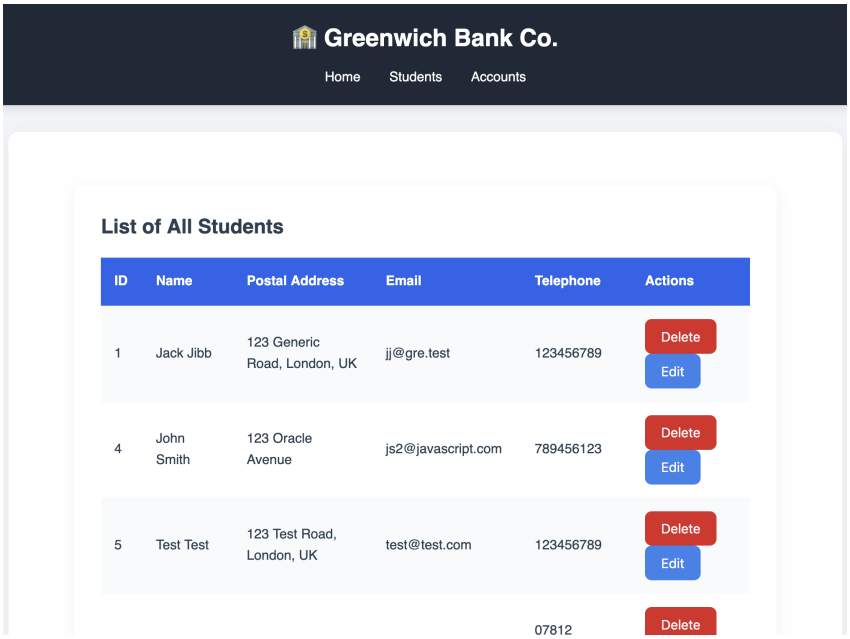

Figure 8: Students tab shows all students in the database, each student can be edited with "edit" button (just a POST form page)

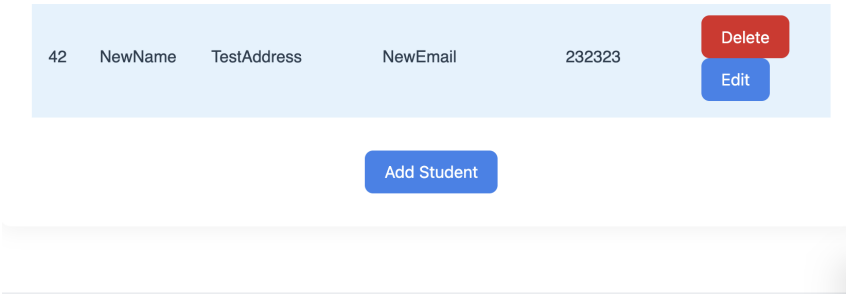

Figure 9: There is an add student button at the bottom of the student list

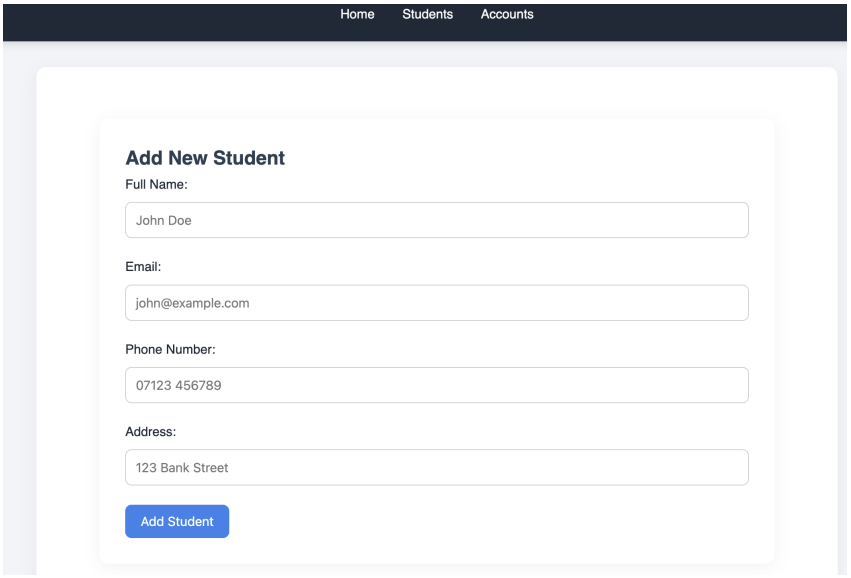

Figure 10: Button navigates to add new student page, which allows user to enter a new student with a POST form

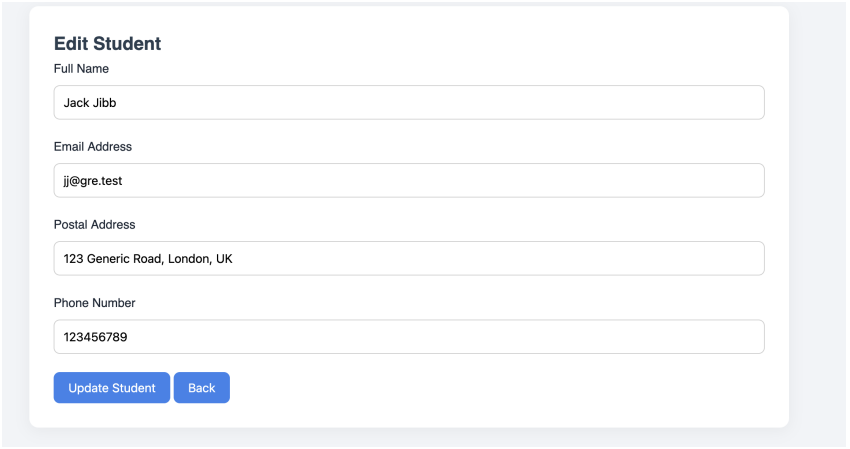

Figure 11: Edit button leads to edit student page, allowing user to update student information

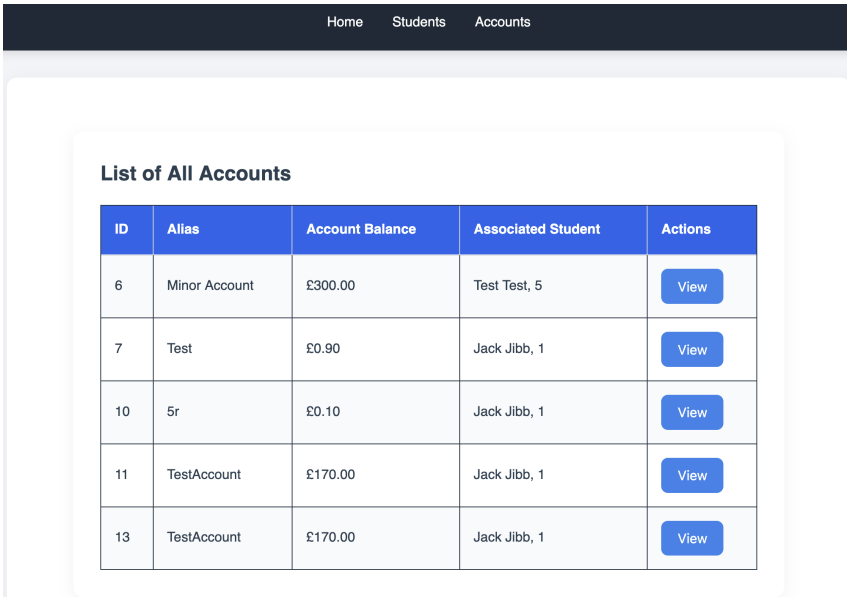

Figure 12: When not ”logged in” Accounts tab shows all accounts, with an edit button

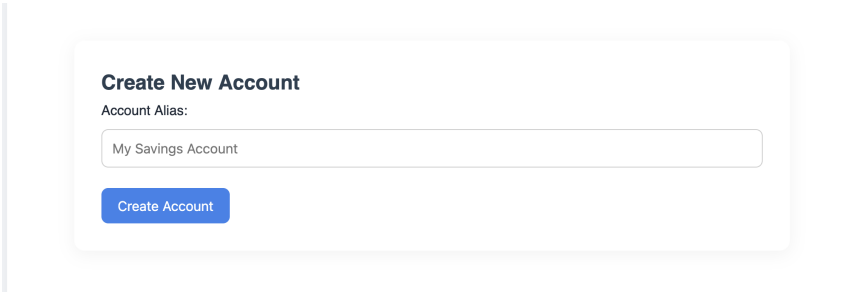

Figure 13: When logged in, user can create an account that associates with the user that is logged in (via cookie)

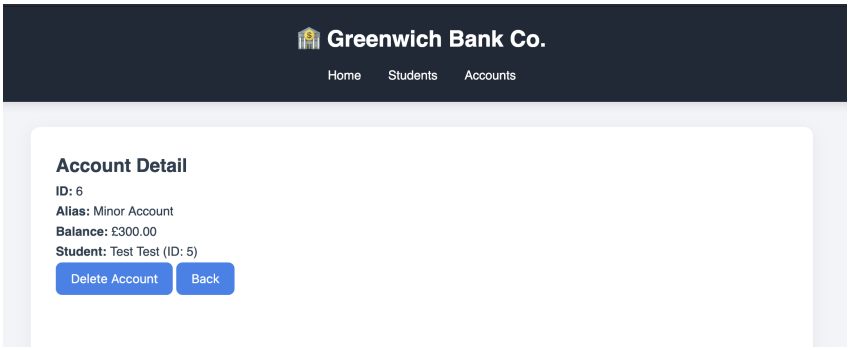

Figure 14: Clicking edit sends user to account details page, where they have option to delete account

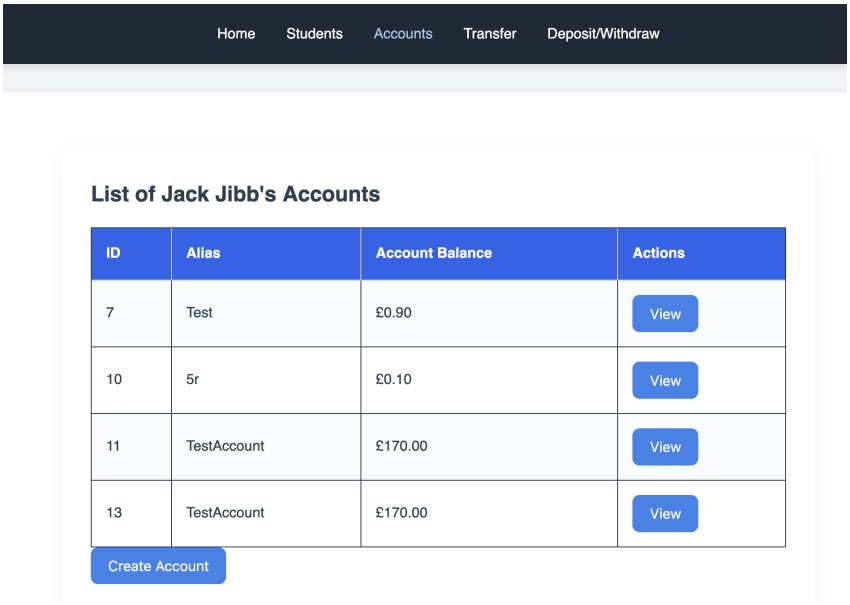

Figure 15: When user logged in they see only their own user accounts

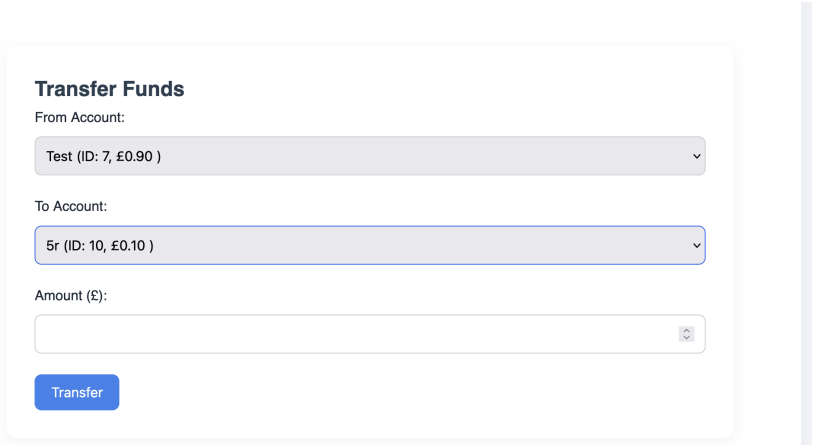

Figure 16: Transfer Form page allows user to transfer from 1 account to another

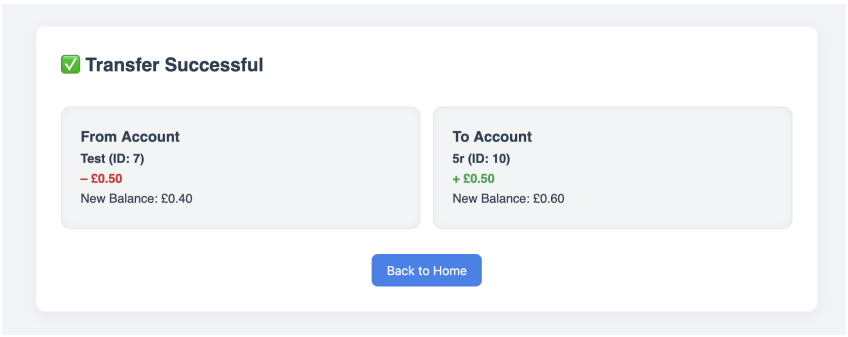

Figure 17: Screen showing successful transfer, displays the transfer data

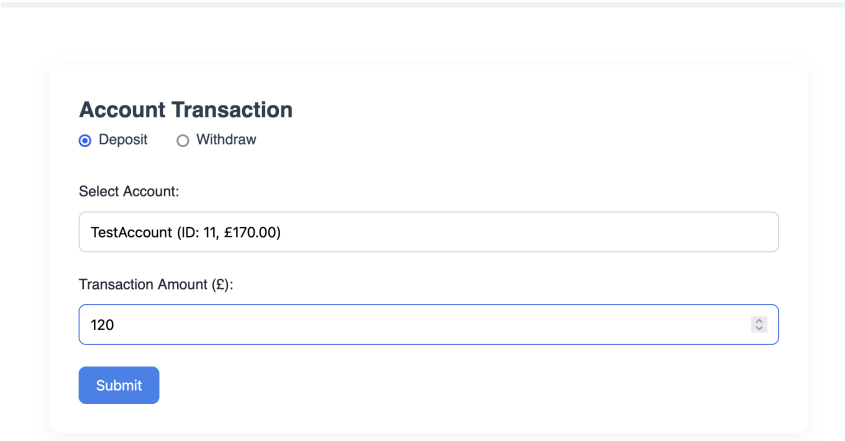

Figure 18: Form for withdrawing or depositing money. Radio buttons choose which operation to perform.

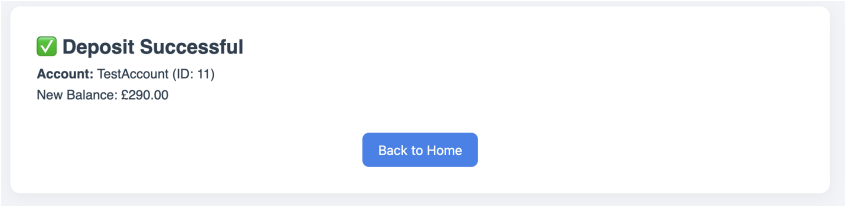

Figure 19: Screen showing a successful deposit (also works for withdraw)

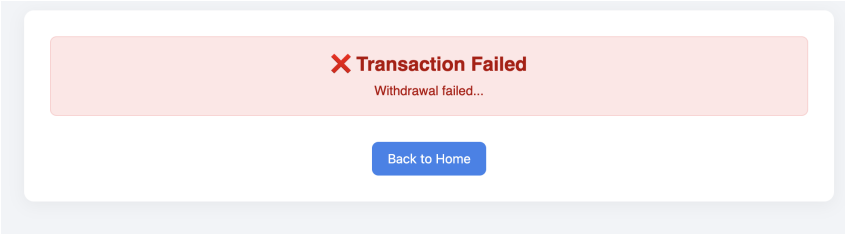

Figure 20: Upon any transaction failure, shows the failure and what transaction failed.

6.1.2 CLI Application Screenshots

Figure 21: CLI main menu, shows which options to choose

```
Enter your choice: 2
=== Create an Account for a Student ===
Enter student ID: 4
Enter account alias: smiths smithy smither
Enter initial account balance: 120
Response Body {"accountAlias":"smiths smithy smither","accountBalance":120.0,"accountID":14}
Account created successfully for student ID: 4
```

Figure 22: Create Account Option


```
Enter your choice: 13
=== Show All Accounts ===
All Accounts: [
  {
    "accountID": 6,
    "studentID": 0,
    "accountAlias": "Minor Account",
    "accountBalance": 300
  },
  {
    "accountID": 7,
    "studentID": 0,
    "accountAlias": "Test",
    "accountBalance": 0.4
  },
  {
    "accountID": 10,
    "studentID": 0,
    "accountAlias": "5r",
    "accountBalance": 0.6
  },
  {
    "accountID": 11,
    "studentID": 0,
    "accountAlias": "TestAccount",
    "accountBalance": 290
  },
  {
    "accountID": 13,
    "studentID": 0,
    "accountAlias": "TestAccount",
    "accountBalance": 350
  }
]
```


Figure 23: Show All Accounts Option

6.2 WildFly 33 Server Configuration

This section outlines the essential configuration settings from the `standalone-full.xml` used in the deployment of the WildFly 33 application server.

6.2.1 Subsystems and Extensions

The configuration includes a comprehensive set of extensions and subsystems to support a variety of Java EE technologies:

- **EJB, JPA, JSF, JAX-RS:** Enabled via `org.jboss.as.ejb3`, `org.jboss.as.jpa`, etc.
- **Security:** Configured through WildFly Elytron with domains `ApplicationDomain` and `ManagementDomain`.
- **Clustering and Caching:** Infinispan containers set for EJBs and Web session management.
- **Logging:** Console and periodic file handlers with custom formatters.

- **Data Sources:**
  - `BankDS`: MySQL datasource connecting to `bank_db` using driver `com.mysql.cj.jdbc.D`

6.2.2 Management and Access Control

- Management is accessible via HTTP interface bound to `management-http` (port 9990).
- Role-based access is enabled using the `SuperUser` role mapped to the local user `$local`.
- Audit logging is available but currently disabled.

6.2.3 Server Interfaces and Socket Bindings

- Interfaces are defined for `management`, `public`, and `unsecure` IP addresses.
- Common ports include:
  - HTTP: 8080
  - HTTPS: 8443
  - AJP: 8009
  - Management HTTP/HTTPS: 9990 / 9993

6.2.4 Deployment and Monitoring

- Deployment scanner monitors the `deployments` directory every 5 seconds.
- Metrics, health, and microprofile extensions are enabled.

6.2.5 Thread and Resource Management

- Default thread pools are defined for concurrency and EJB processing.
- Resource adapters and managed executors are configured under the EE subsystem.

This configuration enables a robust and secure environment suitable for enterprise Java applications, with database integration, secure authentication, clustering, and deployment automation.

6.3 Source Code Structure

Web App Source Code

```
BankApp/
│
├── src/
│   ├── controller/
│   ├── dao/
│   ├── model/
```

```
│   ├── service/
│   └── rest/
├── webapp/
│   ├── jsp/
│   │   ├── accounts/
│   │   ├── home/
│   │   ├── students/
│   │   └── transactions/
│   ├── css/
│   └── js/
└── META-INF/
```

6.4 References

- Jakarta EE Documentation: `https://jakarta.ee`
- WildFly Docs: `https://docs.wildfly.org`
- MySQL Documentation: `https://dev.mysql.com/doc/`